

Basic Shapes Project Report

Emory Bruington | Visual Studio 2026 | Python 3.14

Repository: <https://github.com/l4mont/csc-223-basic-shapes-project>

Project Description

This program models circles, rectangles, and squares. BasicShape defines the common name, area, and calculation interface. Circle and Rectangle supply area formulas. Square inherits Rectangle and keeps all its dimensions equal. A single loop displays two circles, two rectangles, and one square. Unit tests check calculations, invalid input, automatic updates, and inheritance.

Planning

BasicShape inherits from ABC, and `calc_area()` is abstract so incomplete shape classes cannot be created. Dimensions must be positive real numbers; circle coordinates may be negative or zero. Wrong types raise `TypeError`. Zero, negative, or nonfinite dimensions raise `ValueError`. Names must be nonblank strings. Each setter validates before changing state and then updates area.

Class Hierarchy Plan

Class	Base	Protected attributes	Public properties	Area
BasicShape	ABC	<code>_name</code> , <code>_area</code>	<code>name</code> , <code>area</code>	Abstract method
Circle	BasicShape	<code>_x_center</code> , <code>_y_center</code> , <code>_radius</code>	<code>x_center</code> , <code>y_center</code> , <code>radius</code>	pi times radius squared
Rectangle	BasicShape	<code>_length</code> , <code>_width</code>	<code>length</code> , <code>width</code>	length times width
Square	Rectangle	<code>_side</code> ; inherited dimensions	<code>side</code> , <code>length</code> , <code>width</code>	Inherited formula

Design

The formal ABC interface explicitly connects the classes and requires area-calculation behavior. A duck-typed design would only assume that unrelated objects provide the needed properties. Square inherits Rectangle because its area uses the same formula. Its length and width setters both delegate to side, so changing any dimension resizes the entire square.

Implementation

Each class has its own file and imports its base class. Circle and Rectangle call `super().__init__(name)`. Square calls Rectangle's constructor with two equal dimensions. BasicShape starts area at 0.0 without calling a subclass formula. Rectangle delays its first calculation until both dimensions exist; Circle calculates after storing radius.

Properties validate values, preserve state after rejected assignments, and recalculate automatically. The area property is read-only. Circle.`calc_area()` uses `math.pi`; Square reuses Rectangle.`calc_area()`. Setters also reject values whose calculated area becomes infinity or zero. Modules, classes, properties, and methods have docstrings.

Unit Testing

Environment and Strategy

Visual Studio 2026 used its Python 3.14 interpreter. BasicShapes.pyproj selects main.py and unittest discovery in tests. Test Explorer discovered and ran 43 tests in five files. Earlier terminal testing used Python 3.12.14. The terminal command is `python -m unittest discover -s tests -v`.

Test classes inherit `unittest.TestCase`; methods start with `test_`. Tests arrange an object, perform an operation, and check the result. `assertEqual` checks exact values, `assertAlmostEqual` checks decimal areas, and `assertRaises` checks exceptions. Tests cover the abstract class, names, read-only area, invalid dimensions, recalculation, failed assignments, square equality, and mixed-shape processing.

Required Test Results

Test or Feature	Input or Initial State	Expected Result	Actual Result	Pass / Fail	Correction or Comment
Instantiate BasicShape	<code>BasicShape("Shape")</code>	<code>TypeError</code>	<code>TypeError</code>	Pass	Abstract class enforced
Valid Circle	<code>(-2, 0)</code> , radius 4	$16 \times \pi$	50.26548	Pass	Initial area
Zero radius	0	<code>ValueError</code>	<code>ValueError</code>	Pass	Construction rejected
Negative radius	-1	<code>ValueError</code>	<code>ValueError</code>	Pass	Construction rejected
Nonnumeric radius	"2", None, True, 2j	<code>TypeError</code>	<code>TypeError</code>	Pass	Each value tested
Circle radius change	4 to 8	$64 \times \pi$	201.06193	Pass	Automatic update
Circle coordinate change	<code>x=-7</code> , <code>y=3.5</code>	Area unchanged	$16 \times \pi$ retained	Pass	Center updated
Valid Rectangle	10 by 20	Area 200	200	Pass	Initial area
Invalid Rectangle length	-1	<code>ValueError</code>	<code>ValueError</code>	Pass	Width tested too
Rectangle dimension change	Length 10 to 30; separately width 20 to 40	600; 400	600; 400	Pass	Independent cases
Valid Square	Side 10	All dimensions 10	(10, 10, 10)	Pass	Area 100
Square side change	10 to 20	All dimensions 20	(20, 20, 20)	Pass	Area 400
Square length / width assignment	Length 7; separately width 5	All 7; all 5	All 7; all 5	Pass	Areas 49 and 25
Failed dimension assignment	Invalid type or value	Previous state retained	Previous state retained	Pass	All shape classes
Mixed shape list	2 circles, 2 rectangles, 1 square	Common interface; correct areas	All five verified	Pass	No type chain
Full unittest suite	Five test modules	All tests pass	43 passed; 0 failed	Pass	No errors or skips

Problems and Corrections

The initial implementation checked dimensions but not the resulting area. Assigning radius $1e308$ produced infinity; $1e-300$ produced zero. The numeric-range tests expected `ValueError` and an unchanged object, so they failed. The correction checks the prospective area before storing dimensions. The `test_unrepresentable_area` methods for `Circle`, `Rectangle`, and `Square` then passed.

Tests were added after the initial implementation. The failing cases were corrected and rerun, followed by the full suite. Evidence is saved in `evidence/initial-tests.txt` and `evidence/final-tests.txt`. Correction commit: `3bbd9b0` (Fix area overflow and underflow validation).

Final Test Status

43 tests passed; 0 failed. All required abstraction, validation, recalculation, Square-invariant, and polymorphism checks passed. Visual Studio also ran `main.py` and displayed the expected shape areas and dimension updates. The Debug output reported that `MainThread` exited with code 0.

Reflection

1 How does abc make the interface explicit?

`ABC` and `@abstractmethod` declare that every concrete shape must provide or inherit `calc_area()`.

2 What if a direct subclass does not implement calc_area?

It remains abstract. Trying to create an instance raises `TypeError`, as the tests confirm.

3 How would duck typing differ?

Duck typing would accept any object with the expected properties or methods. This project requires an explicit `BasicShape` inheritance relationship.

4 What do Circle and Rectangle inherit?

They inherit name and area properties, name validation, the numeric-validation helper, and the `calc_area` contract. The base constructor creates `_name` and `_area`.

5 Why does Square inherit Rectangle?

It can reuse the length-and-width representation and product formula while adding the equal-dimension rule.

6 Why can Square reuse calc_area?

When length and width both equal side, length times width is side squared.

Reflection Continued

7 How is the Square invariant preserved?

The side setter validates first, then updates side, length, and width together. The inherited dimension setters delegate to side. Invalid updates leave every value unchanged.

8 How does runtime polymorphism work here?

One loop reads each shape's name and area. Calls to `calc_area()` resolve to Circle's or Rectangle's method based on the actual object; Square inherits Rectangle's method. No type-checking chain chooses a formula.

9 How do properties protect dimensions and area?

Setters reject invalid input before modifying the object and recalculate after valid changes. Clients cannot directly assign area.

10 How did unittest improve verification?

Repeatable assertions checked exact expectations, including invalid values and state preservation. These checks found a boundary problem that ordinary demonstration values missed.

11 Which failed test improved the program?

`TestCircle.test_unrepresentable_area` showed that radius `1e308` was accepted even though its area became infinity. Checking the new area before assignment fixed the defect; matching tests cover rectangles and squares.

12 How did docstrings help?

They explain each class's role, valid inputs, exceptions, and automatic recalculation. Square's documentation makes its synchronized resizing behavior clear.

13 What is the main lesson?

Inheritance saves repeated code, but a subclass must preserve its own rules. ABC states the required behavior, properties protect state, and tests check that the pieces work together.